HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference

Shuzhang Zhong[1,2], Yanfan Sun[5], Ling Liang[2], Runsheng Wang[2,3,4], Ru Huang[2,3,4], Meng Li[1,2,4*]
[1]Institute for Artificial Intelligence, Peking University, Beijing, China
[2]School of Integrated Circuits, Peking University, Beijing, China
[3]Institute of Electronic Design Automation, Peking University, Wuxi, China
[4]Beijing Advanced Innovation Center for Integrated Circuits, Beijing, China
[5]School of Computer Science and Engineering, Beihang University, Beijing, China

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133274

***Abstract*—The Mixture of Experts (MoE) architecture has demonstrated significant advantages as it enables to increase the model capacity without a proportional increase in computation. However, the large MoE model size still introduces substantial memory demands, which usually requires expert offloading on resource-constrained platforms and incurs significant overhead. Hybrid CPU-GPU inference has been proposed to leverage CPU computation to reduce expert loading overhead but faces major challenges: on one hand, the expert activation patterns of MoE models are highly unstable, rendering the fixed mapping strategies in existing works inefficient; on the other hand, the hybrid CPU-GPU schedule for MoE is inherently complex due to the diverse expert sizes, structures, uneven workload distribution, etc. To address these challenges, in this paper, we propose HybriMoE, a hybrid CPU-GPU inference framework that improves resource utilization through a novel CPU-GPU scheduling and cache management system. HybriMoE introduces (i) a dynamic intra-layer scheduling strategy to balance workloads across CPU and GPU, (ii) an impact-driven inter-layer prefetching algorithm, and (iii) a score-based caching algorithm to mitigate expert activation instability. We implement HybriMoE on top of the kTransformers framework and evaluate it on three widely used MoE-based LLMs. Experimental results demonstrate that HybriMoE achieves an average speedup of 1.33× in the prefill stage and 1.70× in the decode stage compared to state-of-the-art hybrid MoE inference framework. Our code is available at: https://github.com/PKU-SEC-Lab/HybriMoE.**

I. Introduction

Mixture of Experts (MoE) has emerged as a promising solution to enhance computational efficiency of Large Language Models (LLMs) without compromising model performance [1], [2]. By employing dynamic routing functions that allocate input tokens to a subset of experts, MoE enables the scaling of LLM parameters and capabilities without a proportional increase in computational demands.

Despite its advantages, MoE introduces significant memory requirements, which pose a particular challenge for deployment on edge devices with limited memory resources. To mitigate this issue, expert offloading techniques store expert weights in secondary storage, such as CPU memory or SSDs, and load them into GPU memory through PCIE on demand [3]. In such offloading scenarios, the primary bottleneck becomes the overhead associated with on-demand loading, driven by the large communication scale and limited bandwidth. To mitigate this problem, several studies have explored quantization, prefetching or caching techniques to reduce latency [4]–[7].

Previous works in other offloading scenarios have further explored leveraging CPU computation to reduce the frequency of memory transfers [8], [9]. Techniques such as PowerInfer [10] and Caraserve [11] have achieved notable success by exploiting activation patterns or optimizing adapter usage during inference. Similarly, MoE-specific offloading approaches, including Fiddler [12] and kTransformers [13], utilize the CPU to execute expert layers during cache misses. As illustrated in Figure 1, when a cache miss occurs, the CPU processes the corresponding expert computation instead of transferring the layer to the GPU, reducing data transfer overhead.

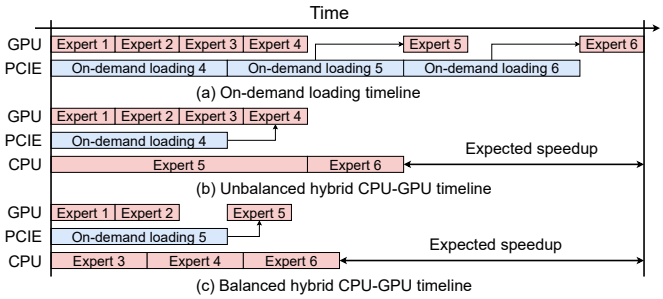

Fig. 1. Execution timeline of three scenarios. Expert computation time on the GPU remains constant, while CPU execution time increases linearly with workload. The balanced scheduling in (c) achieves improved utilization and reduces overall execution time.

While CPU computation is effective for traditional inference tasks, MoE models present unique challenges that complicate their application. Expert activations in MoE models are typically less skewed and exhibit significant variability across iterations, making it difficult to predict which experts will be activated [6]. This dynamic behavior complicates the balancing of workloads between CPU and GPU, as static task allocation strategies fail to adapt to real-time changes in workload distribution. However, existing solutions rely on **fixed mapping strategies** based on historical activation frequencies, neglecting the dynamic and unpredictable nature of MoE inference. These limitations result in suboptimal resource utilization and increased inference latency as illustrated in figure 1(b) and (c).

In light of these challenges and opportunities, we propose **HybriMoE**, a hybrid CPU-GPU scheduling and cache management system to improve the efficiency of MoE inference. Reducing latency in hybrid systems requires maximizing hardware resource utilization, which depends on effective task-to-hardware mapping. However, the dynamic nature of MoE models poses significant challenges to designing optimal mapping strategies. To address this, HybriMoE introduces a comprehensive optimization framework to improve mapping efficiency through three key directions: (i) intra-layer hybrid scheduling, (ii) inter-layer prefetching, and (iii) inter-iteration cache

This work was supported in part by NSFC under Grant 62495102 and Grant 92464104, in part by the National Key Research and Development Program under Grant 2024YFB4505004, in part by Beijing Municipal Science and Technology Program under Grant Z241100004224015, and in part by 111 Project under Grant B18001.

*Corresponding author: meng.li@pku.edu.cn

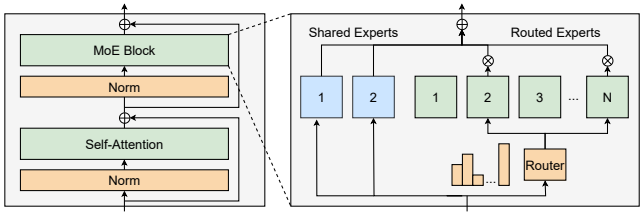

Fig. 2. An example of MoE architecture with shared and routed experts.

TABLE I
COMPARISON WITH EXISTING OFFLOADING WORKS.

	Offload Granularity	CPU Computation	Dynamic Mapping	Cache Optimization
Powerinfer	Neuron	Decode	✓	LFU
llama.cpp	Layer	Prefill+Decode	×	LFU
AdapMoE	Expert	×	✓	LRU
KTrans	Expert	Decode	×	LFU
Ours	Expert	Prefill+Decode	✓	Score-Aware

management. The key contributions of HybriMoE are as follows:

- **Hybrid MoE CPU-GPU Scheduling.** An efficient hybrid scheduling algorithm for MoE inference that dynamically balances workloads across GPUs and CPUs, optimizing resource utilization and minimizing latency through prioritized task execution and data transfer management.
- **Impact-driven prefetching.** A prefetching mechanism that simulates the potential impact of preloading experts from subsequent layers and prioritizes those with the higher expected gains.
- **MoE-specialized Cache Management.** An expert score-based caching strategy that prioritizes high-demand experts across layers to minimize cache misses.
- **System Implementation.** We implement HybriMoE on top of ktransformers framework. We evaluate HybriMoE on three popular MoE-based LLMs and various platforms. Compared to existing hybrid scheduling methods, HybriMoE achieves 1.33× and 1.70x speedup on prefill and decode stages respectively.

II. BACKGROUND

A. Mixture-of-Experts

Mixture-of-Experts (MoE) models offer an efficient solution for handling the computational demands of LLMs by activating only a subset of experts [2], [14]–[16]. Unlike traditional dense networks, MoE models use a gating function G to select which experts process a given input token, reducing the number of active parameters and improving computational efficiency. Given an input x and N experts $E_0, ...E_{N-1}$ the output y of the MoE layer can be expressed as:

$$y = \sum_{i=0}^{N-1} \text{Softmax}(\text{TopK}(x \cdot W_g))_i E_i(x) \quad (1)$$

The total number of experts N and the number of activated experts K vary among different MoE implementations. For instance, the Mixtral model employs 8 experts, with only 2 being active at a time [17]. In contrast, DeepSeek utilizes 64 experts, activating 6 at once. This larger, finer-grained expert pool allows for greater specialization and more efficient knowledge acquisition [18], [19]. Additionally, as shown in figure 2, DeepSeek employs a shared expert strategy, where a subset of experts—known as shared experts—are activated for all tokens. The original experts are defined as routed experts. This reduces redundancy among experts, ensuring efficient processing by minimizing unnecessary computational overlap, thus enhancing overall model efficiency.

B. Efficient MoE Offloading

Parameter-offloading techniques have been proposed to address the significant memory requirements of large language models (LLMs) [20], [21]. However, these techniques are primarily designed for dense models and involve loading or prefetching all parameters, leading to unnecessary communication overhead. To accommodate the sparse activation patterns in Mixture-of-Experts (MoE) models, several specialized techniques have been introduced, including advanced gating, prefetching, and quantization strategies [3]–[7], [22]–[26]. These methods aim to minimize the on-demand loading overhead, reducing unnecessary memory transfers and improving overall performance.

C. Hybrid CPU-GPU Scheduling

Previous offloading techniques have primarily focused on reducing memory transfer overhead by offloading certain computations to the CPU [9]. For instance, PowerInfer [10] reduces GPU memory demand by executing less frequently activated neurons on the CPU, taking advantage of skewed activation patterns. Caraserve [11] addresses cold-start delays in LoRA serving by utilizing CPU assistance and employing rank-aware scheduling to reduce latency. These methods are effective in scenarios where activations are skewed or tasks have long periods of parameter reuse.

In the context of MoE models, techniques like Fiddler [12] and kTransformers [13] extend this concept by offloading expert layer computation to the CPU during cache misses. Specifically, when an expert is not in the GPU cache, the CPU executes the corresponding expert layer instead of loading it from memory. These approaches aim to optimize memory usage by exploiting CPU-GPU parallelism and mitigating the overhead of loading large models onto the GPU.

In table I, we compare HybriMoE with prior-art works qualitatively. As can be observed, HybriMoE features CPU-GPU hybrid scheduling to improve the efficiency of both prefill and decode stages.

III. MOTIVATION

The primary bottleneck in existing hybrid CPU-GPU scheduling for MoE inference is the **suboptimal resource utilization**. To address this issue, we begin by analyzing the main challenges of finding an efficient **mapping strategy**.

Challenge 1: High Instability of MoE Activation Patterns. In existing hybrid CPU-GPU scheduling research, both sparse models with highly skewed activations, like PowerInfer, and dense models (or LoRA inference) exhibit relatively stable activation patterns. In these models, activation is either concentrated on a few 'hot' neurons or remains consistent over time, making scheduling and workload balancing easier. In contrast, MoE models have unpredictable activation patterns, with experts being activated in a dynamic and frequently changing manner. As shown in figure 3(a), compared with neuron-level sparsity, the activation frequency of MoE is more evenly distributed, making it challenging to predict the future expert usage. This lack of stability makes it difficult to determine an optimal CPU-GPU scheduling strategy in advance, leading to suboptimal resource utilization and inefficiency.

Opportunity 1: MoE-specific Cache and Prefetch Optimization. Despite the instability of MoE activations, certain temporal and structural patterns present opportunities for optimization. The temporal correlation of expert activation provides a basis for cache

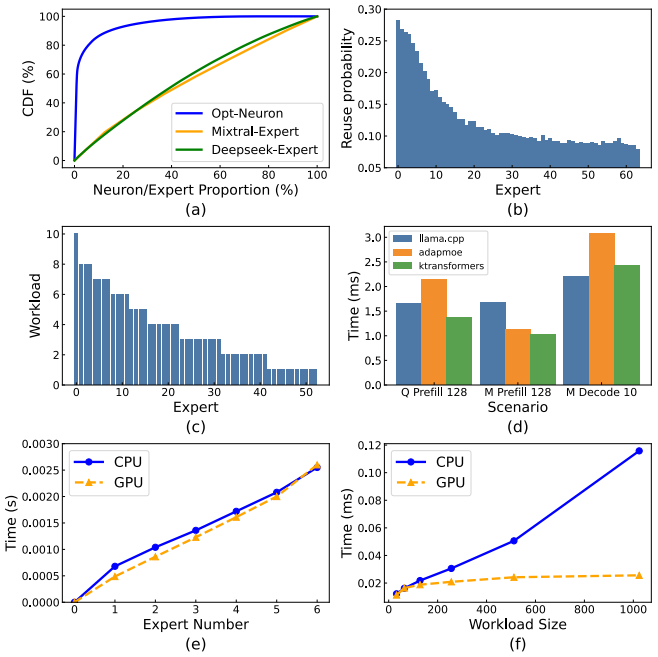

Fig. 3. (a) Cumulative activation frequency(CDF) for neurons and experts, (b) Reuse probability of experts by score, suggesting cache optimization opportunities, (c) Expert workload distribution of DeepSeek in a prefill forward, (d) Latency of prefill 128 tokens for Qwen2(Q), Mixtral(M) and decode 10 tokens for Mixtral with three existing methods, (e) CPU vs. GPU time for varying numbers of experts at fixed load, with CPU benefiting from overlapping computations. (f) CPU and GPU time across workload sizes.

optimization: experts with higher activation scores are more likely to be reused in the next iteration as shown in Figure 3(b), suggesting that retaining high-score experts in cache can reduce access latency. Additionally, MoE models often exhibit high activation similarity between adjacent layers, which can be leveraged for prefetching. These MoE-specific optimizations provide a promising approach to reducing the challenges posed by the dynamic nature of expert activation.

Challenge 2: Complexity of MoE Structure and Dynamic Scheduling. Minimizing latency in MoE inference requires maximizing hardware utilization, but existing fixed-mapping methods often lead to load imbalances and underutilized resources. The scheduling complexity is further increased by the diverse structures of MoE models, with variations in shared expert usage, expert size and number, and runtime cache behavior. Additionally, uneven load distribution and variable execution order in the prefill stage make efficient scheduling even more challenging as shown in figure 3(c). Given the need for layer-by-layer adjustments, a static optimal solution is impractical, making real-time scheduling a significant challenge. As illustrated in figure 3(d), the performance of three existing strategies vary in different stages and models.

Opportunity 2: MoE-specific scheduling rules. Despite the NP-Hard nature of the scheduling problem, in the specific context of MoE inference on CPU-GPU systems, several key observations can guide the design of efficient scheduling rules. First, expert transfer times remain relatively constant, simplifying decision-making. Additionally, GPU computation time scales linearly with the number of activated experts, while CPU computation benefits from overlapping memory access and computation due to its larger cache. As

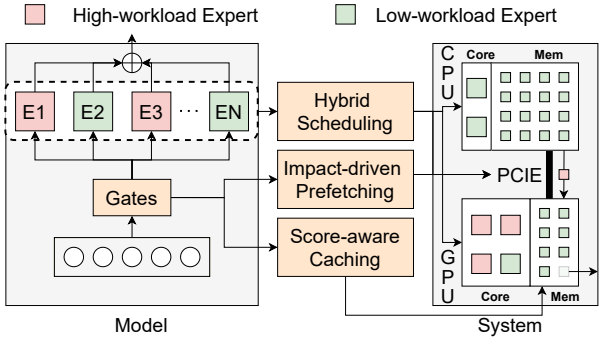

Fig. 4. Overview of HybriMoE.

shown in Figure 3(e), the first expert computation on the CPU is slower, but subsequent tasks are processed faster with better cache utilization. Similarly, Figure 3(f) shows that GPU time remains stable with increasing workload, whereas CPU time grows linearly with workload. Leveraging these patterns, predefined scheduling rules can help achieve efficient workload balancing for MoE models.

IV. HybriMoE Design

A. Overview

This paper introduces HybriMoE, a CPU-GPU hybrid scheduling system tailored for MoE inference on memory-limited devices. HybriMoE addresses the challenges of unbalanced hardware utilization caused by the dynamic activation patterns and structural complexity of MoE models. The system incorporates three key techniques: (i) an efficient hybrid scheduling algorithm that dynamically distributes workloads between GPUs and CPUs, (ii) a score-based expert caching strategy that prioritizes high-demand experts to minimize cache misses, and (iii) an impact-driven prefetching mechanism that predicts and preloads high-demand experts, further enhancing resource utilization and reducing latency.

Figure 4 illustrates the overview of HybriMoE. The system begins with a warmup phase to collect essential performance metrics, such as CPU and GPU processing speeds and data transfer latency. During inference, HybriMoE leverages this information to implement hybrid CPU-GPU scheduling, score-aware caching, and impact-driven prefetching, ensuring efficient task execution and optimized resource usage throughout the inference process.

B. Hybrid Scheduling Strategy

The scheduling problem in MoE inference is inherently complex due to the dynamic nature of expert activation and the need to balance workloads across heterogeneous resources. To address these challenges, HybriMoE proposes a hybrid scheduling strategy that simplifies the task-to-hardware mapping by introducing three key priority rules:

- **GPU Priority**: The GPU prioritizes the computation of cached experts, executing higher-load experts first.
- **CPU Priority**: The CPU prioritizes the computation of uncached experts, focusing on lower-load tasks for efficient execution. Additionally, the CPU can process cached experts when the CPU is idle, following a low-to-high load order.
- **Transfer Priority**: The CPU-GPU transfer mechanism prioritizes the movement of high-load uncached experts from CPU to GPU to minimize computation delays.

These rules constrain the ordering of experts on devices, simplifying the scheduling problem into an allocation problem:

$$\arg\min_{cpu\_expert, gpu\_expert} \max(CPU_{TIME}(cpu\_expert), GPU_{TIME}(gpu\_expert)) \quad (2)$$

This formulation does not account for the finish time of data transfers, as expert loading must be completed before GPU computation begins.

Based on these priority rules, HybriMoE divides all activated experts into a GPU queue and a CPU queue. The GPU queue contains cached experts on the GPU, sorted by load in descending order. The CPU queue contains uncached experts on the CPU, sorted by load in ascending order.

Before the actual execution, HybriMoE performs a simulation phase to evaluate scheduling strategies and identify an efficient task allocation plan tailored to the specific workload. This simulation approximates the execution process by iteratively filling the CPU computation, GPU computation, and data transferring timelines, enabling the system to determine a scheduling configuration that minimizes overall latency while balancing resource utilization across heterogeneous hardware.

During each step of the simulation, the system selects the timeline with the earliest completion time and executes the corresponding operation—either a computation task on the CPU or GPU, or a data transfer via PCIE. Task selection adheres to the scheduling priorities: the GPU prioritizes high-load cached experts, the CPU focuses on low-load uncached experts and, when its queue is empty, processes low-load cached experts from the GPU queue, while PCIE prioritizes high-load uncached experts for faster availability on the GPU.

If an expert is transferred from the CPU to the GPU, it is inserted into the GPU queue in descending order of load, ensuring high-load tasks are prioritized for GPU computation. This iterative simulation continues until all experts are computed, effectively modeling the execution process and testing different scheduling strategies.

The scheduling process is illustrated through an example in figure 5. In this scenario, the GPU computation time is assumed to be constant, the CPU computation time is proportional to the expert's load, and the transmission time is fixed at 3 units. The scheduling algorithm in HybriMoE identifies an optimal strategy where the CPU computes the cached expert E while the GPU processes the uncached expert C, effectively improving hardware utilization by balancing workloads across resources.

C. Impact-driven prefetching

Due to the residual connections in LLMs, hidden states across consecutive layers exhibit a high degree of similarity, making expert prefetching an effective method to optimize resource utilization. While several existing works have adopted prefetching mechanisms, none of them discuss the critical trade-offs involved when multiple subsequent layers' experts can be prefetched. Specifically, these works do not explore how to strategically decide which layer's experts should be prioritized for prefetching to maximize resource efficiency.

Inspired by the scheduling algorithm in IV-B, we propose impact-driven prefetching. Before executing a prefetch, the system performs a simulation to evaluate the potential gains of prefetching specific experts. This simulation estimates the impact of preloading a given expert on overall scheduling efficiency, allowing the algorithm to prioritize experts that yield the highest resource utilization improvements. The greedy nature of this simulation ensures minimal computational overhead, making it practical for real-time inference scenarios.

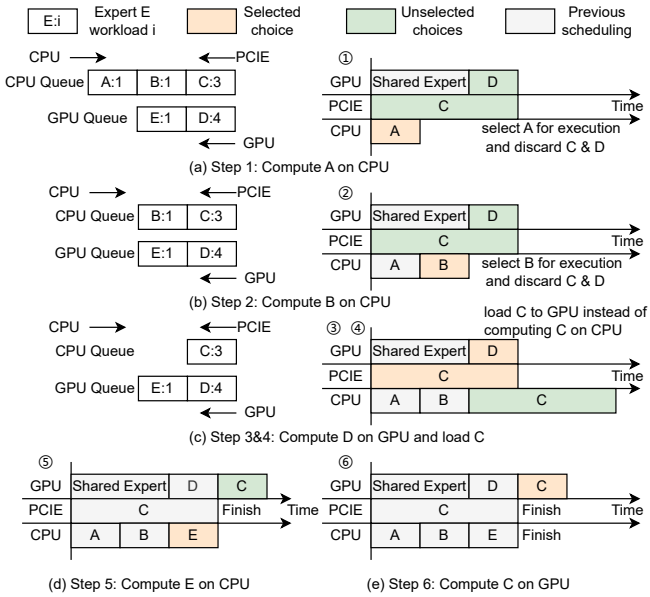

Fig. 5. An example of hybrid scheduling. The CPU computes the cached expert E while the GPU computes the uncached expert C to achieve better hardware utilization.

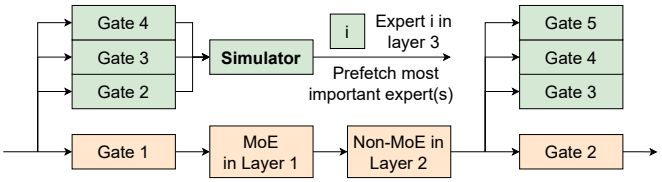

Fig. 6. Impact-driven prefetch workflow.

Specifically, HybriMoE predicts expert activations for the next three layers by reusing the gating information from those layers as illustrated in figure 6. This prediction guides the prefetching mechanism, enabling the system to efficiently preload experts likely to be activated in subsequent computations.

D. Score-aware Caching

Traditionally, the Least Frequently Used (LFU) and Least Recently Used (LRU) algorithms have been employed for MoE cache management. However, these strategies fail to account for the specific activation patterns observed in MoE models, where expert scores provide valuable predictive signals for future activations. As discussed in III, not only are currently activated experts likely to be reused in the future, but experts with high scores that were not activated also exhibit a higher probability of being selected in subsequent iterations.

To leverage this insight, we propose Score-Aware Caching, a novel cache replacement strategy tailored for MoE models. Specifically, we introduce the **Minus Recent Score (MRS)** replacement policy, which prioritizes retaining experts based on their routing scores.

Define s as the routing scores of all experts in the current iteration, S as the estimated priority score, α as the averaging coefficient, the update of the estimated priority can be expressed as :

$$S = \alpha \times TopP(s) + (1 - \alpha) \times S \quad (3)$$

Here, only the top p expert scores will be accumulated. This is derived from the observation in figure 3(b) that the reuse probability

TABLE II
CONFIGURATION OF THREE EVALUATED MOE MODELS.

	Mixtral	Qwen2	DeepSeek
#Layers	32	28	26
#Shared Experts	0	1	2
#Routed Experts	8	64	64
#Activated Experts	2	8	6
Shared Expert Size	/	(3584, 20480)	(2048, 1408)
Routed Expert Size	(4096, 14336)	(3584, 18944)	(2048, 1408)

of experts with lower scores does not exhibit significant differences. Typically, we set p to twice the number of activated experts.

V. SYSTEM IMPLEMENTATION

We implement the HybriMoE system on top of the kTransformers framework and llama.cpp kernels. KTransformers provides a flexible infrastructure for kernel injection, enabling seamless support for hybrid CPU-GPU execution. To optimize the system workflow, we incorporate parallel execution across CPU, GPU, and PCIe transfers, utilizing fine-grained CUDA stream scheduling for efficient resource management. Additionally, we modify the C++ kernels to handle expert computation task allocation directly, minimizing redundant Python overhead and improving execution efficiency. For quantization, we leverage Marlin quantization, a state-of-the-art 4-bit quantization kernel from llama.cpp, to significantly enhance computational efficiency and reduce memory usage.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

1) Platforms: We evaluate HybriMoE on the NVIDIA RTX A6000. For the CPU, we utilize an Intel Xeon Gold 5220R processor, restricting usage to 10 cores to simulate real-world edge deployment scenarios with limited resources. To assess the system's performance and scalability under varying hardware configurations, we adjust the upper bound of the GPU expert cache ratio.

2) Models: We evaluate our system using three widely adopted MoE models with distinct characteristics: Mixtral-8x7B-Instruct [17] (Mixtral), DeepSeek-V2-Lite-Chat [18] (DeepSeek), and Qwen2-57B-A14B-Instruct [16] (Qwen2). As summarized in Table II, these models differ in the number and size of experts, as well as their architectural configurations. Mixtral represents MoE models with a smaller number of larger experts, while Qwen2 and DeepSeek are representative of models with a larger number of smaller experts. Notably, Qwen2 and DeepSeek also incorporate shared experts, which are activated for all input tokens.

3) Baselines: We evaluate HybriMoE against three representative open-source MoE inference frameworks: llama.cpp [27], AdapMoE [5], and kTransformers [13], each representing a distinct scheduling approach. llama.cpp is a CPU-GPU hybrid scheduling baseline that statically maps model layers to CPU or GPU. AdapMoE is the SOTA for GPU-centric MoE scheduling, minimizing on-demand loading overhead through adaptive prefetching and caching. kTransformers is the SOTA for CPU-GPU hybrid MoE scheduling, mapping high-activation-frequency experts (e.g., shared experts) to the GPU to maximize efficiency.

4) Metrics: Auto-regressive decoding consists of two stages: the prefill stage and the decoding stage. We evaluate the performance of HybriMoE separately for these two stages. For the prefill stage, we use Time To First Token (TTFT), which measures the latency from receiving the input prompt to generating the first token. For the decoding stage, we use Time Between Tokens (TBT), which captures the time taken to generate each subsequent token. These metrics provide a clear assessment of both initial latency and sustained efficiency during inference.

5) Datasets: For the prefill stage, we evaluate performance under varying input lengths by sampling traces of different lengths from multiple datasets, including MT Bench [28], Vicuna Bench [29] and ChatGPT Prompts [30]. In contrast, for the decoding stage, as performance is not sensitive to input length, we use only the ChatGPT Prompts dataset to evaluate the TBT metric.

TABLE III
MOE INFERENCE SPEEDUP BREAKDOWN OF PROPOSED TECHNIQUES.

	Technique	Latency(s)	Speedup
Prefill	Baseline	1.47	
	Baseline+Scheduling	1.17	1.26×
	Baseline+Prefetching	1.39	1.06×
	All	1.13	1.31×
Decode	Baseline	0.21	
	Baseline+Scheduling	0.14	1.46×
	Baseline+Prefetching	0.18	1.15×
	Baseline+Caching	0.15	1.38×
	All	0.11	1.86×

B. End-to-End Performance

1) Prefill Stage: We evaluate the prefill stage performance of HybriMoE by comparing it against three baselines: llama.cpp, AdapMoE, and kTransformers. Figure 7 presents the TTFT results across various input legnths(around 32, 128, 512 and 1024 tokens) and different GPU expert cache ratios(25%, 50% and 75%).

HybriMoE demonstrates consistent improvements over the baselines across all input lengths and cache configurations. llama.cpp exhibits significantly higher prefill latency due to its naive static mapping strategy, which allocates entire layers of experts to the CPU. This approach fails to balance workloads effectively, particularly in the prefill stage where computational demand is high, leading to substantial delays. Compared to kTransformers, HybriMoE achieves an average speedup of **1.33**× across different input lengths and cache configurations. This improvement is driven by HybriMoE's hybrid scheduling and impact-driven prefetching mechanisms, which dynamically balance workloads and reduce cache misses, enabling more efficient resource utilization.

2) Decode Stage: Figure 8 illustrates the decode performance results for three MoE models. HybriMoE consistently achieves the highest throughput across all cache ratios and models, demonstrating its ability to dynamically balance workloads and fully utilize hardware resources during the decode stage. Compared to kTransformers, HybriMoE achieves an average throughput improvement of **1.70**×. Also, it is worth noting that llama.cpp demonstrates relatively strong performance in this stage, especially compared to its prefill stage results. This is primarily due to the smaller computational load per expert in the decode stage, which allows CPU-based computation to proceed faster. Additionally, the impact of uneven expert mapping is less pronounced compared to the prefill stage, and the overall resource overhead remains low, favoring llama.cpp's static scheduling strategy in this specific context.

C. Ablation Study

We further explore how the components of our method contribute to the result. Performance was measured for Qwen2 under 25%

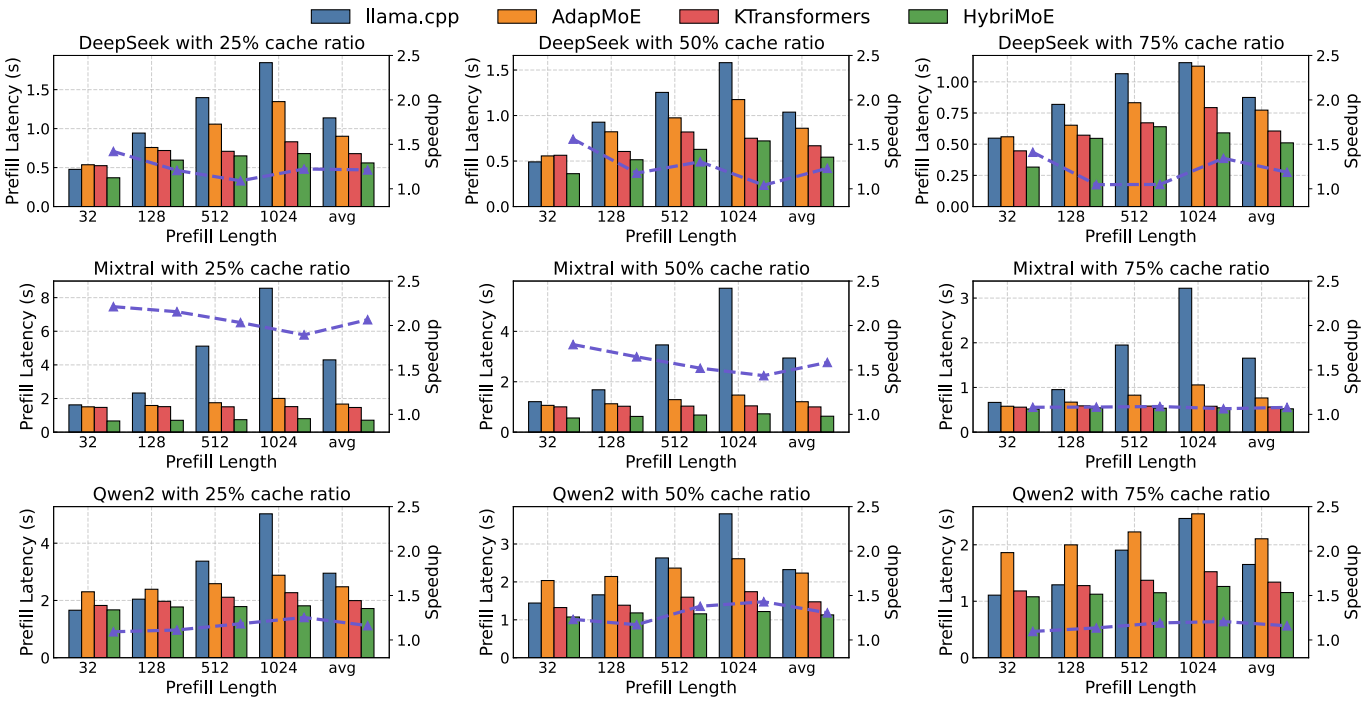

Fig. 7. Prefill stage performance comparison across different input lengths and cache ratios, highlighting relative speedups against kTransformers.

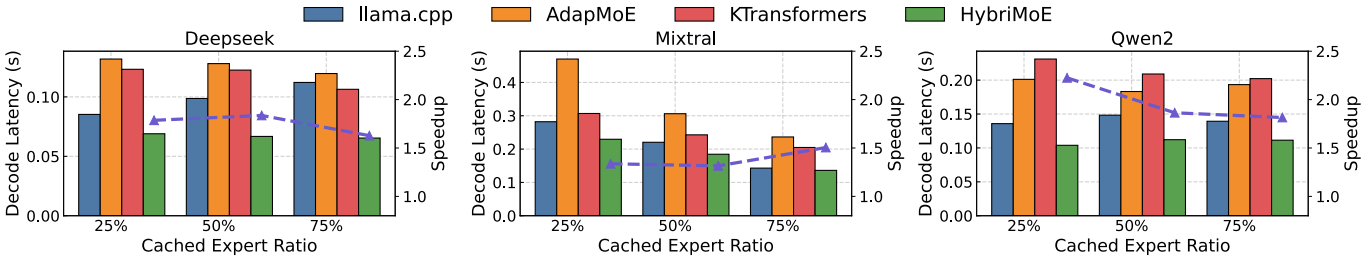

Fig. 8. Decode stage performance comparison across different cache ratios.

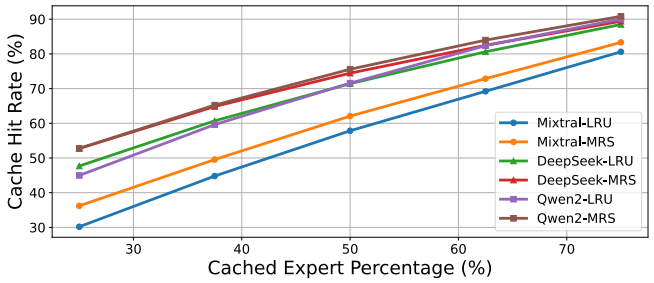

Fig. 9. Cache Hit Rate Comparison Between MRS and LRU Across Different Cached Expert Percentages.

expert cache ratio for the two stages. The baseline is ktransformers framework. The results are illustrated in table III.

D. Discussions

1) Score-aware Cache Management Analysis: Figure 9 compares the cache hit rates of HybriMoE's Minus Recent Score (MRS) strategy and the traditional Least Recently Used (LRU) strategy across three models under varying cached expert percentages. At 25% cache capacity, MRS outperforms LRU by 6% to 8%, with Mixtral improving from 30.2% to 36.2%, DeepSeek from 47.7% to 52.7%, and Qwen2 from 45.0% to 52.8%. As cache capacity increases to 75%, the gap narrows (e.g., Mixtral: 83.3% vs. 80.6%), as higher capacities reduce expert competition, diminishing the relative impact of the caching strategy. These results highlight MRS's effectiveness, particularly under limited cache settings.

VII. CONCLUSION

This paper presents HybriMoE, a hybrid CPU-GPU scheduling and cache management system designed to address the challenges of MoE inference, including dynamic expert activations and workload imbalances. By incorporating dynamic intra-layer scheduling, impact-driven prefetching, and score-aware caching, HybriMoE achieves efficient resource utilization and reduced latency. Experiments on various MoE models demonstrate that HybriMoE achieves an average speedup of 1.33x in prefill latency and 1.70x in decode latency compared to state-of-the-art methods. These results highlight HybriMoE's effectiveness in optimizing hybrid MoE inference and its potential for scalable deployment on resource-constrained devices.

REFERENCES

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017.

[2] S. Masoudnia and R. Ebrahimpour, "Mixture of experts: a literature survey," *Artificial Intelligence Review*, vol. 42, pp. 275–293, 2014.

[3] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[4] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, M. Yang, and M. Rhu, "Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference," *arXiv preprint arXiv:2308.12066*, 2023.

[5] S. Zhong, L. Liang, Y. Wang, R. Wang, R. Huang, and M. Li, "Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference," *arXiv preprint arXiv:2408.10284*, 2024.

[6] X. Song, Z. Zhong, and R. Chen, "Promoe: Fast moe-based llm serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[7] P. Tang, J. Liu, X. Hou, Y. Pu, J. Wang, P.-A. Heng, C. Li, and M. Guo, "Hobbit: A mixed precision expert offloading system for fast moe inference," *arXiv preprint arXiv:2411.01433*, 2024.

[8] J. You, J. Wu, X. Jin, and M. Chowdhury, "Ship compute or ship data? why not both?" in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 633–651.

[9] D. Park and B. Egger, "Improving throughput-oriented llm inference with cpu computations," in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, 2024, pp. 233–245.

[10] Y. Song, Z. Mi, H. Xie, and H. Chen, "Powerinfer: Fast large language model serving with a consumer-grade gpu," *arXiv preprint arXiv:2312.12456*, 2023.

[11] S. Li, H. Lu, T. Wu, M. Yu, Q. Weng, X. Chen, Y. Shan, B. Yuan, and W. Wang, "Caraserve: Cpu-assisted and rank-aware lora serving for generative llm inference," *arXiv preprint arXiv:2401.11240*, 2024.

[12] K. Kamahori, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: Cpu-gpu orchestration for fast inference of mixture-of-experts models," *arXiv preprint arXiv:2402.07033*, 2024.

[13] KVCache-AI, "Ktransformers: A flexible framework for experiencing cutting-edge llm inference optimizations," 2024, https://github.com/kvcache-ai/ktransformers.

[14] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[15] M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard *et al.*, "No language left behind: Scaling human-centered machine translation," *arXiv preprint arXiv:2207.04672*, 2022.

[16] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.

[17] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[18] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[19] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[20] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, "Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.

[21] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.

[22] J. Li, Q. Su, Y. Yang, Y. Jiang, C. Wang, and H. Xu, "Adaptive gating in mixture-of-experts based language models," *arXiv preprint arXiv:2310.07188*, 2023.

[23] Y. Wei, J. Du, J. Jiang, X. Shi, X. Zhang, D. Huang, N. Xiao, and Y. Lu, "Aptmoe: Affinity-aware pipeline tuning for moe models on bandwidth-constrained gpu nodes," in *2024 SC24: International Conference for High Performance Computing, Networking, Storage and Analysis SC*. IEEE Computer Society, 2024, pp. 1436–1449.

[24] P. Li, X. Jin, Y. Cheng, and T. Chen, "Examining post-training quantization for mixture-of-experts: A benchmark," *arXiv preprint arXiv:2406.08155*, 2024.

[25] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "Moe-infinity: Activation-aware expert offloading for efficient moe serving," *arXiv preprint arXiv:2401.14361*, 2024.

[26] X. He, S. Zhang, Y. Wang, H. Yin, Z. Zeng, S. Shi, Z. Tang, X. Chu, I. Tsang, and O. Y. Soon, "Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference," 2024. [Online]. Available: https://arxiv.org/abs/2410.17954

[27] G. Gerganov, "ggerganov/llama.cpp: Port of facebook's llama model in c/c++," 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[28] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[30] MohamedRashad, "Chatgpt-prompts," 2023. [Online]. Available: https://huggingface.co/datasets/MohamedRashad/ChatGPT-prompts